说一下对Spring的理解

Spring框架核心特性包括：

- IOC容器：Spring通过控制反转实现了对象的创建和对象间依赖关系的管理，从而降低系统耦合度
- AOP：面向切面编程，允许开发者定义横切关注点，例如事务管理、安全控制等，这些独立于业务逻辑的代码。
  通过AOP，可以将这些关注点模块化，提高代码的可维护性和可重用性
- 事务管理：Spring提供了一致的事务管理接口，支持声明式和编程式事务。
  开发者可以轻松的进行事务管理，而无需关注**具体的事务API**
- MVC框架：Spring MVC是一个基于Servlet API构建的Web框架，采用了模型-视图-控制器架构，它支持灵活的URL到页面控制器的映射，以及多种视图技术

Spring核心思想?

核心思想	解决的核心问题	实现机制	典型应用场景
IoC（控制反转）	降低对象创建与依赖管理的耦合	BeanFactory / ApplicationContext 容器统一管理Bean生命周期	组件解耦、模块化开发、可插拔设计
DI（依赖注入）	消除依赖关系的硬编码	构造器注入 / Setter注入 / 注解注入（@Autowired）	Service依赖DAO、数据源注入
AOP（面向切面编程）	横切逻辑与业务逻辑分离	动态代理（JDK / CGLIB）+ 切面配置	日志、事务、权限控制、缓存

Spring IoC和AOP介绍一下?

IoC（控制反转）

IoC，全称 Inversion of Control

传统开发中，对象之间的依赖由开发者通过 `new` 关键字手动创建和维护，导致类与类之间高度耦合。IoC 的核心理念是将对象的创建权交给**IoC 容器**来统一管理，开发者只需声明依赖，容器负责实例化和注入，从而大幅降低模块间的耦合度。

依赖注入（DI，Dependency Injection）是实现 IoC 思想的主要方式。

AOP（面向切面编程）

AOP，全称 Aspect-Oriented Programming，是一种**横向抽取公共逻辑的编程范式。**

它将与核心业务无关、但被多个模块共同使用的通用逻辑（如日志记录、事务管理、权限校验等）从业务代码中剥离出来，统一封装为"切面"，在需要的地方动态织入，从而减少重复代码，降低模块间耦合度。

Spring AOP 底层基于**动态代理**实现：

- 若目标对象**实现了接口**，使用 **JDK 动态代理**创建代理对象；
- 若目标对象**未实现接口**，则使用 **Cglib** 生成目标类的子类作为代理对象。

IoC 与 AOP 的协同使用

IoC 和 AOP 在 Spring 框架中相辅相成：IoC 负责管理对象及其依赖关系，AOP 负责将横切关注点动态切入业务逻辑。两者结合，可以让代码职责更加单一，实现更清晰的分层架构与模块化管理。

典型场景举例： 通过 IoC 容器统一管理 Service 层和 DAO 层的依赖关系，同时借助 AOP 在 Service 层透明地织入事务控制、日志记录等横切功能，使业务代码更加简洁、易于维护。

为什么用jdk动态代理的类必须实现接口?

这是由 **JDK 动态代理的底层机制**决定的。

根本原因

JDK 动态代理生成的代理类，本质上是在运行时动态生成一个**新的类**，这个类会：

1. **继承 `Proxy` 类**
2. **实现目标对象的接口**

由于 Java 是单继承的，代理类已经继承了 `Proxy`，就**无法再继承目标类本身**了。所以它只能通过**实现接口**的方式，来保证代理类和目标类有相同的方法签名，从而实现方法拦截。

用一句话总结

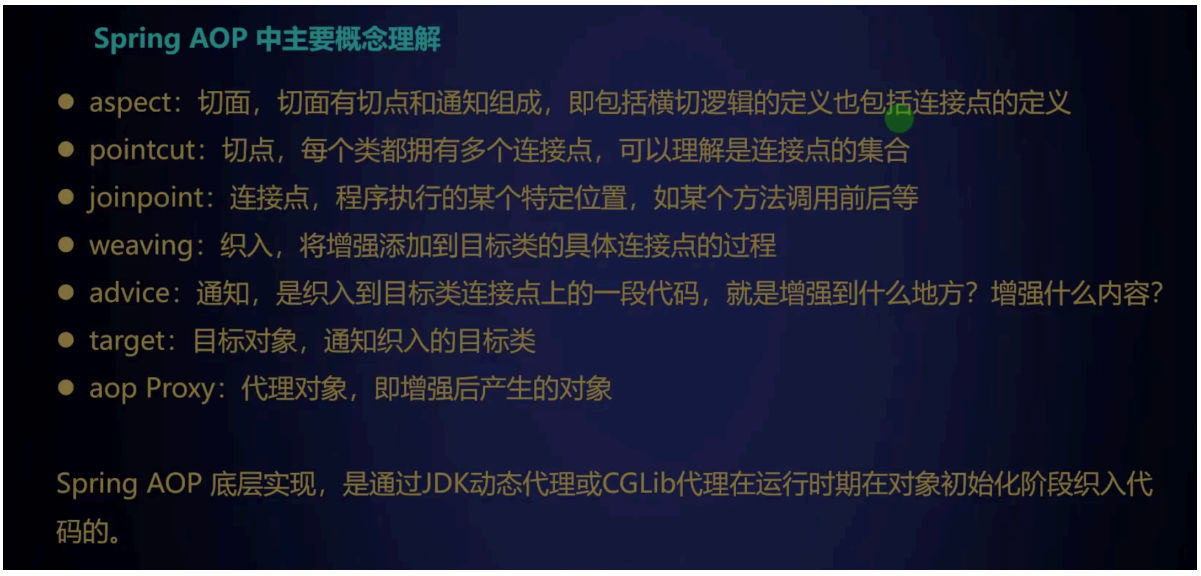
> JDK 动态代理是"基于接口"的代理，它生成的代理类和你的目标类共同实现同一个接口，调用方通过接口调用代理，代理再转发给真实对象。如果目标类没有接口，代理类就不知道该"长什么样"，自然无法生成。

对比 Cglib

Cglib 则是直接**继承目标类**，生成一个子类作为代理，所以不需要接口。但也因此，如果目标类或方法是 `final` 的，Cglib 同样无能为力——因为 `final` 类/方法不能被继承和重写。

	JDK 动态代理	Cglib
代理方式	实现接口	继承目标类
要求	目标类必须有接口	目标类不能是 final
性能	反射调用，稍慢	字节码直接调用，稍快

Spring AOP




```
@Aspect
@Component
public class MyAspect {

    private Logger logger = LoggerFactory.getLogger(this.getClass());

    /*
    *  定义pointcut 切点
    * */
    @Pointcut(value = "execution(* com.nozomi.aop.controller.*.*(..))")
    public void pointCut(){}

    /*
    *  定义advice 通知
    * */
    @Around(value = "pointCut()")
    public Object around(ProceedingJoinPoint joinPoint) throws Throwable{
        String methodName = joinPoint.getSignature().getName();
        String className = joinPoint.getTarget().getClass().getName();
```

```
        ObjectMapper m = new ObjectMapper();
        Object[] args = joinPoint.getArgs();
        logger.info("类{}调用方法{},参数:{}",className,methodName,
m.writeValueAsString(args));
        Object result = joinPoint.proceed();
        logger.info("类{}调用方法{},参数:{},返回结果:{}",className,methodName,
m.writeValueAsString(args),m.writeValueAsString(result));
        return result;
    }

}
```

整个类是一个切面
*连接点，指程序执行过程中的一个点，也就是程序执行过程中**所有可能被拦截的点**（在 Spring AOP 中，仅指**方法执行**这个点）。*
*切点，**匹配连接点的规则 / 表达式**，作用是从所有连接点中筛选出需要被增强的那些点。*

实现机制

Spring AOP的实现依赖于动态代理技术。动态代理是在运行时动态生成代理对象，而不是在编译时。它允许开发者在运行时指定要代理的接口和行为，从而实现在不修改源码的情况下增强方法的功能。

Spring AOP支持两种动态代理：

- 基于JDK的动态代理：使用java.lang.reflect.Proxy类和java.lang.reflect.InvocationHandler接口实现。这种方式需要代理的类实现一个或多个接口。
- 基于CGLIB的动态代理：当被代理的类没有实现接口时，Spring会使用CGLIB库生成一个被代理类的子类作为代理。

主要解决什么问题

Spring AOP 是面向对象编程的补充，主要用于解决 跨多个类的 “弱共性” 逻辑（如日志、事务、权限校验） 的统一处理问题，避免这些非核心功能侵入业务代码或破坏继承体系。

它通过 切面（Aspect）定义增强逻辑（Advice），再用切点（Pointcut） 精准匹配目标方法，最后将增强逻辑织入到目标方法执行流程中，实现无侵入式的功能增强。

常见注解

常用的注解包括：

- @Aspect：用于定义切面，标注在切面类上。
- @Pointcut：定义切点，标注在方法上，用于指定连接点。
- @Before：在方法执行之前执行通知。
- @After：在方法执行之后执行通知。
- @Around：在方法执行前后都执行通知。
- @AfterReturning：在方法执行后返回结果后执行通知。
- @AfterThrowing：在方法抛出异常后执行通知。
- @Advice：通用的通知类型，可以替代@Before、@After等。

Spring IoC

概念

IOC：Inversion Of Control，即控制反转，是一种设计思想。在传统的 Java SE 程序设计中，我们直接在对象内部通过 new 的方式来创建对象，是程序主动创建依赖对象；而在Spring程序设计中，IOC 是有专门的容器去控制对象。

所谓控制*就是对象的创建、初始化、销毁*

实现机制

- 反射：Spring IoC容器通过**反射机制 动态地加载类、创建对象实例**及**调用对象方法**
- 依赖注入：IOC（控制反转）是 Spring 的核心思想，而依赖注入（DI）是实现 IOC 的核心方式：由 Spring 容器统一管理应用程序中各组件的依赖关系，开发者可通过构造函数注入、Setter 注入（属性注入）或方法注入的方式，将组件间的依赖关系通过配置文件或注解的形式告知 Spring 容器。
- 工厂模式（设计模式）：Spring IOC 容器基于工厂模式设计，核心职责是统一负责 Bean 的实例化、依赖注入以及整个生命周期的管理，而非由开发者手动控制 Bean 的创建和生命周期。

- 容器实现：Spring IOC容器是实现IOC的核心，通常使用BeanFactory或ApplicationContext来管理Bean。BeanFactory是IOC容器的基本形式，提供基本的IOC功能；ApplicationContext是BeanFactory的扩展，并提供更多企业级功能。

控制反转、依赖注入、依赖倒置分别是什么?

- 控制反转：“控制”指的是对程序执行流程的控制，而“反转”指的是在没有使用框架之前，程序员自己控制整个程序的执行。在使用框架之后，整个程序的执行流程通过框架来控制。流程的控制权从程序员“反转”给了框架。
- 依赖注入：依赖注入和控制反转恰恰相反，它是一种具体的编码技巧。我们不通过 new 的方式在类内部创建依赖类的对象，而是将依赖的类对象在外部创建好之后，通过构造函数、函数参数等方式传递（或注入）给类来使用。
- 依赖倒置：这条原则跟控制反转有点类似，主要用来指导框架层面的设计。高层模块不依赖低层模块，它们共同依赖同一个抽象。抽象不要依赖具体实现细节，具体实现细节依赖抽象。

依赖注入了解吗？怎么实现依赖注入的?

在传统编程中，当一个类需要使用另一个类的对象时，通常会在该类内部通过`new`关键字来创建依赖对象，这使得类与类之间的耦合度较高。

而依赖注入则是将对象的创建和依赖关系的管理交给 Spring 容器来完成，类只需要声明自己所依赖的对象，容器会在运行时将这些依赖对象注入到类中，从而降低了类与类之间的耦合度，提高了代码的可维护性和可测试性。

具体到Spring中，常见的依赖注入的实现方式，比如构造器注入、Setter方法注入，还有字段注入。

- **构造器注入：**通过构造函数传递依赖对象，保证对象初始化时依赖已就绪。

```java
@Service
public class UserService {
    private final UserRepository userRepository;

    // 构造器注入（Spring 4.3+ 自动识别单构造器，无需显式@Autowired）
    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }
}
```

- **Setter 方法注入：**通过 Setter 方法设置依赖，灵活性高，但依赖可能未完全初始化。

```java
public class PaymentService {
    private PaymentGateway gateway;

    @Autowired
    public void setGateway(PaymentGateway gateway) {
        this.gateway = gateway;
    }
}
```

- **字段注入：**直接通过 `@Autowired` 注解字段，代码简洁但隐藏依赖关系，不推荐生产代码。

```java
@Service
public class OrderService {
    @Autowired
    private OrderRepository orderRepository;
}
```

如果让你设计一个SpringIoc，你觉得会从哪些方面考虑这个设计?

- Bean的生命周期管理：需要设计Bean的创建、初始化、销毁等生命周期管理机制，可以考虑使用工厂模式和单例模式来实现。
- Bean的作用域：需要支持多种Bean作用域，比如单例、原型、会话、请求等，可以考虑使用Map来存储不同作用域的Bean实例。
- 依赖注入：需要实现依赖注入的功能，包括属性注入、构造函数注入、方法注入等，可以考虑使用反射机制和XML配置文件来实现。
- 异常处理：需要考虑异常处理机制，包括Bean创建异常、依赖注入异常等，可以考虑使用try-catch机制来处理异常。
- 配置文件加载：需要支持从不同的配置文件中加载Bean的相关信息，可以考虑使用XML、注解或者Java配置类来实现。

动态代理和静态代理的区别

代理是一种常用的设计模式，目的是：为其他对象提供一个代理以控制对某个对象的访问，将两个类的关系解耦。代理类和委托类都要实现相同的接口，因为代理真正调用的是委托类的方法。

区别：

- 静态代理：由程序员创建或者是由特定工具创建，在代码编译时就确定了被代理的类是一个静态代理。静态代理通常只代理一个类；
- 动态代理：在代码运行期间，运用反射机制动态创建生成。动态代理代理的是一个接口下的多个实现类。

能使用静态代理的方式实现AOP吗?

当然**可以**用静态代理实现 AOP 的基本功能，比如你在代码里手动写个代理类，在目标方法前后加日志或者事务控制，技术上完全可行。但问题是，静态代理在实际生产中基本没人用，因为它有三大硬伤：

- **第一是代码爆炸**。比如你有 100 个 Service 类需要加事务，就得写 100 个对应的静态代理类，里面全是重复的 `try-catch` 提交回滚代码，维护起来简直是灾难。
- **第二是僵化**。一旦业务接口改了个方法名，所有相关的代理类都得跟着改，而动态代理通过反射调用目标方法，根本不怕这种变动。
- **第三是无法动态筛选**。比如你想只给带 `@Transactional` 注解的方法加事务，静态代理只能写死逻辑，而 Spring AOP 可以在运行时通过切点表达式精准匹配需要增强的方法。

所以 Spring 才选了 JDK 动态代理和 CGLIB：它们能在**运行时动态生成代理类**，一个切面配置就能覆盖成百上千个方法。像事务管理这种全局需求，用静态代理手动绑定根本不可行。

什么是反射，有哪些使用场景?

概念

反射机制是指程序在运行状态下，对于任意一个类，都能够获取这个类的所有属性和方法；对于任意一个对象，都能够调用它的任意属性和方法。也就是说，Java 反射允许在运行时获取类的信息并动态操作对象

反射特性

反射具有以下特性：

1. **运行时类信息访问**：反射机制允许程序在运行时获取类的完整结构信息，包括类名、包名、父类、实现的接口、构造函数、方法和字段等。
2. **动态对象创建**：可以使用反射API动态地创建对象实例，即使在编译时不知道具体的类名。这是通过Class类的newInstance()方法或Constructor对象的newInstance()方法实现的。
3. **动态方法调用**：可以在运行时动态地调用对象的方法，包括私有方法。这通过Method类的invoke()方法实现，允许你传入对象实例和参数值来执行方法。
4. **访问和修改字段值**：反射还允许程序在运行时访问和修改对象的字段值，即使是私有的。这是通过Field类的get()和set()方法完成的。

使用场景

> 1、Spring框架的依赖注入（DI）和控制反转（IoC）

Spring 使用反射来实现其核心特性：依赖注入。

在Spring中，开发者可以通过XML配置文件或者基于注解的方式声明组件之间的依赖关系。当应用程序启动时，Spring容器会扫描这些配置或注解，然后利用反射来实例化Bean（即Java对象），并根据配置自动装配它们的依赖。

例如，当一个Service类需要依赖另一个DAO类时，开发者可以在Service类中使用@Autowired注解，而无需自己编写创建DAO实例的代码。Spring容器会在运行时解析这个注解，通过反射找到对应的DAO类，实例化它，并将其注入到Service类中。这样不仅降低了组件之间的耦合度，也极大地增强了代码的可维护性和可测试性。

2.动态代理的实现

在需要对现有类的方法调用进行拦截、记录日志、权限控制或事务管理等场景中，反射结合动态代理被广泛运用

以 Spring AOP 为例，它通过定义切面（Aspect），能将日志、事务等通用功能作为横切关注点，在不修改业务代码的情况下，自动植入到目标方法中。

> 例如，为了给所有的服务层方法添加日志记录功能，可以定义一个切面，在这个切面中，Spring会使用JDK动态代理或CGLIB（如果目标类没有实现接口）来创建目标类的代理对象。这个代理对象在调用任何方法前或后，都会执行切面中定义的代码逻辑（如记录日志），而这一切都是在运行时通过反射来动态构建和执行的，无需硬编码到每个方法调用中。

Spring的事务在什么情况下会失效?

1. **异常被捕获**：如果事务方法内部使用 try-catch 将异常捕获且没有重新抛出，Spring 的 AOP 代理层感知不到任何异常，会认为方法正常执行完毕并提交事务，导致事务失效，数据库操作不会回滚。
2. **受检异常未配置回滚**：Spring 默认只对 `RuntimeException` 及其子类和 `Error` 进行回滚，对受检异常（如 `IOException`、`SQLException`）默认不回滚而是直接提交事务。如果业务中抛出的是受检异常，需要通过 `@Transactional(rollbackFor = Exception.class)` 显式指定，否则事务失效。
3. **事务传播属性设置不正确**：如果在多个事务之间存在事务嵌套，且事务传播属性配置不正确，可能会导致事务失效。
4. **多数据源的事务管理**: 如果在使用多数据源时，事务管理没有正确配置或者存在多个 @Transactional 注解时，可能会导致事务失效。
5. **事务在非公开方法中失效**: 如果 @Transactional 注解标注在私有方法上或者非 public 方法上，事务也会失效。

事务传播属性

事务传播属性解决的问题是：**当一个有事务的方法调用另一个方法时，被调用的方法该怎么处理事务?** 是加入已有事务，还是新开一个，还是不用事务?

Spring 用 `@Transactional(propagation = ...)` 来配置，共有 7 种：

最常用的几种:

`REQUIRED`（默认）：有事务就加入，没有就新建。最常见，99% 的场景用这个。

```
// 外层有事务，内层加入同一个事务，一起提交或回滚
@Transactional(propagation = Propagation.REQUIRED)
public void inner() { ... }
```

`REQUIRES_NEW`：不管外层有没有事务，都新开一个独立事务，外层事务挂起等待。内外事务互不影响，内层回滚不影响外层，外层回滚也不影响内层。

```
// 典型场景：记录操作日志，不管业务成不成功日志都要保存
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void saveLog() { ... }
```

`NESTED`：在外层事务内创建一个嵌套事务（savepoint），内层回滚只回到 savepoint，不影响外层。但外层回滚会连带内层一起回滚。

`NEVER`：不允许在事务中运行，如果外层有事务就抛异常。

`NOT_SUPPORTED`：不支持事务，如果外层有事务就把它挂起。

`SUPPORTS`：有事务就用，没有就不用，完全随外层。

`MANDATORY`：强制要求外层必须有事务，没有就抛异常。

最容易踩坑的场景

就是图中第3条说的，**同一个类内部方法互相调用**：

```
@Service
public class OrderService {

    @Transactional
    public void outer() {
        // 直接调用同类的 inner，不走代理
        this.inner();
    }

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void inner() { ... }
}
```

这里 `inner()` 的传播属性完全失效，因为 Spring 事务是基于 AOP 代理实现的，同类内部直接用 `this` 调用绕过了代理，Spring 根本感知不到这次调用，传播属性自然不起作用。

多数据源事务失效

正常单数据源时，Spring 有一个默认的事务管理器 `DataSourceTransactionManager`，`@Transactional` 自动找它用，没有歧义。

但多数据源时，你有多个事务管理器，`@Transactional` 不知道该用哪个，就会出问题。

举个例子

假设系统里有两个数据源，一个主库存订单，一个副库存日志：

```
@Configuration
public class DataSourceConfig {

    @Bean
    @Primary
    public PlatformTransactionManager orderTransactionManager() {
        return new DataSourceTransactionManager(orderDataSource());
    }

    @Bean
    public PlatformTransactionManager logTransactionManager() {
        return new DataSourceTransactionManager(logDataSource());
    }
}
```

然后有个业务方法，想同时操作两个库：

```java
@Service
public class OrderService {

    @Autowired
    private OrderMapper orderMapper;    // 操作主库

    @Autowired
    private LogMapper logMapper;        // 操作副库

    @Transactional  // 没有指定用哪个事务管理器
    public void createOrder() {
        orderMapper.insert(order);    // 主库操作
        logMapper.insert(log);        // 副库操作
        throw new RuntimeException(); // 模拟异常
    }
}
```

这里的问题是：`@Transactional` 没指定事务管理器，默认会用标了 `@Primary` 的 `orderTransactionManager`，所以它**只管主库的事务。**

异常发生后，主库的订单回滚了，但副库的日志因为根本不在这个事务管理器的管辖范围内，**已经提交了，回不去了。**

正确做法

需要明确指定每个方法用哪个事务管理器：

```java
@Transactional("orderTransactionManager")
public void createOrder() {
    orderMapper.insert(order);
}

@Transactional("logTransactionManager")
public void saveLog() {
    logMapper.insert(log);
}
```

但这样又带来新问题：两个方法用的是不同事务管理器，**它们之间无法保证原子性**，一个成功一个失败是完全可能的。

根本矛盾

这才是多数据源事务的核心难题：**普通的 `@Transactional` 天生只能管一个数据源**，它做不到跨库的原子性。如果真的需要跨库事务，要么接受最终一致性（消息队列补偿），要么引入分布式事务方案比如 Seata。

@Primary作用

`@Primary` 的作用是：当 Spring 容器中同一个类型有**多个 Bean** 时，标记哪一个是**首选**的，注入时有歧义就用它。

没有 @Primary 时会怎样

```
@Bean
public PlatformTransactionManager orderTransactionManager() { ... }

@Bean
public PlatformTransactionManager logTransactionManager() { ... }
```

这时候如果你这样注入：

```
@Autowired
private PlatformTransactionManager transactionManager; // 报错!
```

Spring 发现有两个 `PlatformTransactionManager`，不知道注哪个，直接抛异常。

加了 @Primary

```
@Bean
@Primary
public PlatformTransactionManager orderTransactionManager() { ... }

@Bean
public PlatformTransactionManager logTransactionManager() { ... }
```

再注入时 Spring 发现有歧义，但看到其中一个标了 `@Primary`，就默认用它，不报错。

解决歧义的方式不止一种

`@Primary` 是在**定义 Bean 时**声明优先级，还有另一种方式是在**注入时**明确指定用哪个：

```
@Autowired
@Qualifier("logTransactionManager") // 明确指定用这个
private PlatformTransactionManager transactionManager;
```

两者的区别是：`@Primary` 是全局默认，`@Qualifier` 是局部指定。如果同时用了，`@Qualifier` 优先级更高。

Spring的事务使用this调用，是否生效?

不能生效。因为Spring事务是通过代理对象来控制的，只有通过代理对象的方法调用才会应用事务管理的相关规则。当使用this直接调用时，是绕过了Spring的代理机制，因此不会应用事务设置。

Bean的生命周期

五步骤版本

1. 实例化
2. 依赖注入
3. 初始化
4. 使用 Bean
5. 销毁 Bean

七步骤版本

1. 实例化
2. 依赖注入
3. **BeanPostProcessor before 方法**
4. 初始化
5. **BeanPostProcessor after 方法**
6. 使用 Bean
7. 销毁 Bean

十步骤版本

1. 实例化
2. 依赖注入
3. **BeanNameAware、BeanFactoryAware 方法执行**
4. **BeanPostProcessor before 方法**
5. **InitializingBean 接口方法执行**
6. 初始化（init-method）
7. **BeanPostProcessor after 方法**
8. 使用 Bean
9. 销毁（DisposableBean）

10. 销毁（destroy-method）

Bean是否为单例

> Spring 中的 Bean 默认都是单例的。
>
> 就是说，每个Bean的实例只会被创建一次，并且会被存储在Spring容器的缓存中，以便在后续的请求中重复使用。这种单例模式可以提高应用程序的性能和内存效率。
>
> 但是，Spring也支持将Bean设置为多例模式，即每次请求都会创建一个新的Bean实例。要将Bean设置为多例模式，可以在Bean定义中通过设置scope属性为"prototype"来实现。
>
> 需要注意的是，虽然Spring的默认行为是将Bean设置为单例模式，但在一些情况下，使用多例模式是更为合适的，例如在创建状态不可变的Bean或有状态Bean时。此外，需要注意的是，如果Bean单例是有状态的，那么在使用时需要考虑线程安全性问题。

Bean的单例和非单例，生命周期是否一样

不一样的，Spring Bean 的生命周期完全由 IoC 容器控制。Spring 只帮我们管理单例模式 Bean 的完整生命周期，对于 `prototype` 的 Bean，Spring 在创建好交给使用者之后，则不会再管理后续的生命周期。

具体区别如下：

阶段	单例（Singleton）	非单例（如Prototype）
创建时机	容器启动时创建（或首次请求时，取决于配置）。	每次请求时创建新实例。
初始化流程	完整执行生命周期流程（属性注入、Aware接口、初始化方法等）。	每次创建新实例时都会完整执行生命周期流程（仅到初始化完成）。
销毁时机	容器关闭时销毁，触发 `DisposableBean` 或 `destroy-method`。	**容器不管理销毁**，需由调用者自行释放资源（Spring不跟踪实例）。
内存占用	单实例常驻内存，高效但需注意线程安全。	每次请求生成新实例，内存开销较大，需手动管理资源释放。
适用场景	无状态服务（如Service、DAO层）。	有状态对象（如用户会话、临时计算对象）。

Spring Bean的作用域有哪些?

Spring框架中的Bean作用域（Scope）定义了Bean的生命周期和可见性。不同的作用域影响着Spring容器如何管理这些Bean的实例，包括它们如何被创建、如何被销毁以及它们是否可以被多个用户共享。

Spring支持几种不同的作用域，以满足不同的应用场景需求。以下是一些主要的Bean作用域：

- **Singleton（单例）**：在整个应用程序中只存在一个 Bean 实例。默认作用域，Spring 容器中只会创建一个 Bean 实例，并在容器的整个生命周期中共享该实例。
- **Prototype（原型）**：每次请求时都会创建一个新的 Bean 实例。次从容器中获取该 Bean 时都会创建一个新实例，适用于状态非常瞬时的 Bean。
- **Request（请求）**：每个 HTTP 请求都会创建一个新的 Bean 实例。仅在 Spring Web 应用程序中有效，每个 HTTP 请求都会创建一个新的 Bean 实例，适用于 Web 应用中需求局部性的 Bean。
- **Session（会话）**：Session 范围内只会创建一个 Bean 实例。该 Bean 实例在用户会话范围内共享，仅在 Spring Web 应用程序中有效，适用于与用户会话相关的 Bean。
- **Application**：当前 ServletContext 中只存在一个 Bean 实例。仅在 Spring Web 应用程序中有效，该 Bean 实例在整个 ServletContext 范围内共享，适用于应用程序范围内共享的 Bean。
- **WebSocket（Web套接字）**：在 WebSocket 范围内只存在一个 Bean 实例。仅在支持 WebSocket 的应用程序中有效，该 Bean 实例在 WebSocket 会话范围内共享，适用于 WebSocket 会话范围内共享的 Bean。
- **Custom scopes（自定义作用域）**：Spring 允许开发者定义自定义的作用域，通过实现 Scope 接口来创建新的 Bean 作用域。

在Spring配置文件中，可以通过`<bean>`标签的scope属性来指定Bean的作用域。例如：

```
<bean id="myBean" class="com.example.MyBeanClass" scope="singleton"/>
```

在Spring Boot或基于Java的配置中，可以通过@Scope注解来指定Bean的作用域。例如：

```java
@Bean
@Scope("prototype")
public MyBeanClass myBean() {
    return new MyBeanClass();
}
```

Spring容器中存的是什么?

在Spring容器中，存储的**主要是Bean对象**。

Bean是Spring框架中的基本组件，用于表示应用程序中的各种对象。当应用程序启动时，Spring容器会根据配置文件或注解的方式创建和管理这些Bean对象。Spring容器会负责创建、初始化、注入依赖以及销毁Bean对象。

在Spring中，在bean加载/销毁前后，如果想实现某些逻辑，可以怎么做

在Spring框架中，如果你希望在Bean加载（即实例化、属性赋值、初始化等过程完成后）或销毁前后执行某些逻辑，你可以使用Spring的生命周期回调接口或注解。这些接口和注解允许你定义在Bean生命周期的关键点执行的代码。

使用 init-method 和 destroy-method

在XML配置中，你可以通过 `init-method` 和 `destroy-method` 属性来指定Bean在**初始化阶段**（依赖注入完成后、BeanPostProcessor after之前）和销毁前需要调用的方法。

```
<bean id="myBean" class="com.example.MyBeanClass"
      init-method="init" destroy-method="destroy"/>
```

然后，在你的Bean类中实现这些方法：

```
public class MyBeanClass {

    public void init() {
        // 初始化逻辑，在依赖注入完成后、AOP代理生成前执行
    }

    public void destroy() {
        // 销毁逻辑，在容器关闭时执行
    }
}
```

> 实现InitializingBean和DisposableBean接口

你的Bean类可以实现org.springframework.beans.factory.InitializingBean和org.springframework.beans.factory.DisposableBean接口，并分别实现afterPropertiesSet和destroy方法。

```java
import org.springframework.beans.factory.DisposableBean;
import org.springframework.beans.factory.InitializingBean;

public class MyBeanClass implements InitializingBean, DisposableBean {

    @Override
    public void afterPropertiesSet() throws Exception {
        // 初始化逻辑
    }

    @Override
    public void destroy() throws Exception {
        // 销毁逻辑
    }
}
```

> 使用@PostConstruct和@PreDestroy注解

```java
import javax.annotation.PostConstruct;
import javax.annotation.PreDestroy;

public class MyBeanClass {

    @PostConstruct
    public void init() {
        // 初始化逻辑
    }

    @PreDestroy
    public void destroy() {
        // 销毁逻辑
    }
}
```

> 使用@Bean注解的initMethod和destroyMethod属性

在基于Java的配置中，你还可以在@Bean注解中指定initMethod和destroyMethod属性。

```java
@Configuration
public class AppConfig {

    @Bean(initMethod = "init", destroyMethod = "destroy")
    public MyBeanClass myBean() {
        return new MyBeanClass();
    }
}
```

Spring给我们提供了很多扩展点，有了解吗?

Spring框架提供了许多扩展点，使得开发者可以根据需求定制和扩展Spring的功能。以下是一些常用的扩展点：

1. BeanFactoryPostProcessor：允许在Spring容器实例化bean之前修改bean的定义。常用于修改bean属性或改变bean的作用域。
2. BeanPostProcessor：可以在bean实例化、配置以及初始化之后对其进行额外处理。常用于代理bean、修改bean属性等。
3. PropertySource：用于定义不同的属性源，如文件、数据库等，以便在Spring应用中使用。
4. ImportSelector和ImportBeanDefinitionRegistrar：用于根据条件动态注册bean定义，实现配置类的模块化。
5. Spring MVC中的HandlerInterceptor：用于拦截处理请求，可以在请求处理前、处理中和处理后执行特定逻辑。
6. Spring MVC中的ControllerAdvice：用于全局处理控制器的异常、数据绑定和数据校验。
7. Spring Boot的自动配置：通过创建自定义的自动配置类，可以实现对框架和第三方库的自动配置。
8. 自定义注解：创建自定义注解，用于实现特定功能或约定，如权限控制、日志记录等。

例子

1. BeanFactoryPostProcessor

在Bean实例化之前修改Bean的定义，比如动态修改某个Bean的属性值：

```
@Component
public class MyBeanFactoryPostProcessor implements BeanFactoryPostProcessor {
    @Override
    public void postProcessBeanFactory(ConfigurableListableBeanFactory
beanFactory) {
        // 拿到Bean的定义，修改它的属性
        BeanDefinition bd = beanFactory.getBeanDefinition("orderService");
        bd.setScope("prototype"); // 把作用域改成prototype
    }
}
```

---

2. BeanPostProcessor

Bean初始化前后进行额外处理，AOP代理就是这里生成的：

```
@Component
public class MyBeanPostProcessor implements BeanPostProcessor {

    @Override
    public Object postProcessBeforeInitialization(Object bean, String beanName) {
        System.out.println(beanName + " 初始化之前执行");
        return bean;
    }

    @Override
    public Object postProcessAfterInitialization(Object bean, String beanName) {
        System.out.println(beanName + " 初始化之后执行");
        // 可以在这里返回代理对象替换原始bean
        return bean;
    }
}
```

---

3. PropertySource

自定义属性源，比如从数据库读取配置：

```
@Configuration
@PropertySource("classpath:application.properties") // 从文件加载
public class AppConfig {

    @Value("${db.url}")
    private String dbUrl;
}
```

```
// 也可以自定义属性源从数据库读配置
public class DbPropertySource extends PropertySource<DataSource> {
    @Override
    public Object getProperty(String name) {
        // 从数据库查配置值
        return jdbcTemplate.queryForObject(
            "select value from config where key=?", String.class, name);
    }
}
```

---

4. ImportSelector 和 ImportBeanDefinitionRegistrar

根据条件动态决定注册哪些Bean，Spring Boot自动配置底层就用了这个：

```
// ImportSelector：返回要注册的配置类名
public class MyImportSelector implements ImportSelector {
    @Override
    public String[] selectImports(AnnotationMetadata metadata) {
        // 根据条件决定导入哪些配置类
        if (isWindowsOS()) {
            return new String[]{"com.example.WindowsConfig"};
        }
        return new String[]{"com.example.LinuxConfig"};
    }
}

// ImportBeanDefinitionRegistrar：直接操作BeanDefinition注册表
public class MyRegistrar implements ImportBeanDefinitionRegistrar {
    @Override
    public void registerBeanDefinitions(AnnotationMetadata metadata,
                                        BeanDefinitionRegistry registry) {
        // 手动注册一个Bean
        RootBeanDefinition bd = new RootBeanDefinition(OrderService.class);
        registry.registerBeanDefinition("orderService", bd);
    }
}

// 使用
@Import(MyImportSelector.class)
@Configuration
public class AppConfig { }
```

---

5. HandlerInterceptor

拦截 HTTP 请求，常用于登录校验、日志记录：

```
@Component
public class LoginInterceptor implements HandlerInterceptor {

    @Override
    public boolean preHandle(HttpServletRequest request,
                             HttpServletResponse response, Object handler) {
        // 请求进来时执行，返回false则拦截不放行
```

```
        String token = request.getHeader("token");
        if (token == null) {
            response.setStatus(401);
            return false;
        }
        return true;
    }

    @Override
    public void postHandle(HttpServletRequest request,
                           HttpServletResponse response, Object handler,
                           ModelAndView modelAndView) {
        // Controller执行完、视图渲染前执行
    }

    @Override
    public void afterCompletion(HttpServletRequest request,
                                HttpServletResponse response, Object handler,
                                Exception ex) {
        // 整个请求完成后执行，常用于清理资源
    }
}

// 注册拦截器
@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(new LoginInterceptor())
                .addPathPatterns("/**")
                .excludePathPatterns("/login");
    }
}
```

6. ControllerAdvice

全局统一处理异常，避免每个Controller都写try-catch：

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    // 处理业务异常
    @ExceptionHandler(BusinessException.class)
    public Result handleBusinessException(BusinessException e) {
        return Result.fail(e.getCode(), e.getMessage());
    }

    // 处理所有未捕获的异常
    @ExceptionHandler(Exception.class)
    public Result handleException(Exception e) {
        return Result.fail(500, "服务器内部错误");
    }
}
```

7. Spring Boot 自动配置

自定义 starter，让框架自动配置：

```
// 自定义自动配置类
@Configuration
@ConditionalOnClass(OssClient.class) // classpath有这个类才生效
@EnableConfigurationProperties(OssProperties.class)
public class OssAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean // 用户没有自定义才注册
    public OssClient ossClient(OssProperties properties) {
        return new OssClient(properties.getEndpoint(), properties.getKey());
    }
}

// resources/META-INF/spring.factories 中注册
org.springframework.boot.autoconfigure.EnableAutoConfiguration=\
  com.example.OssAutoConfiguration
```

---

8. 自定义注解

比如自定义权限校验注解：

```
// 定义注解
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface RequireRole {
    String value(); // 需要的角色
}

// 用AOP拦截注解
@Aspect
@Component
public class RoleAspect {

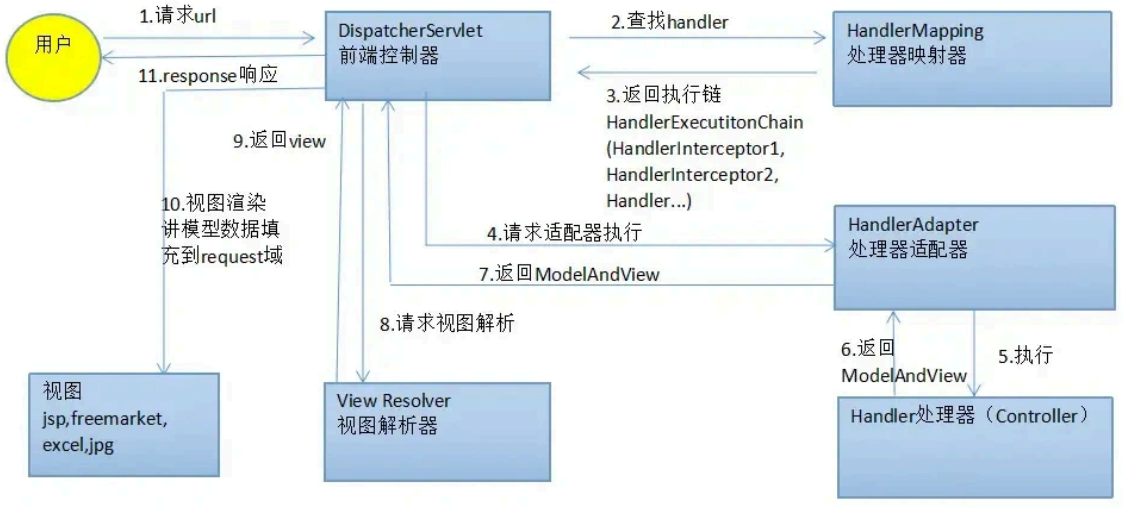
    @Around("@annotation(requireRole)")
    public Object check(ProceedingJoinPoint pjp, RequireRole requireRole) throws
Throwable {
        String role = requireRole.value();
        String userRole = getCurrentUserRole();
        if (!userRole.equals(role)) {
            throw new UnauthorizedException("无权限");
        }
        return pjp.proceed();
    }
}

// 使用
@RequireRole("admin")
@GetMapping("/delete")
public Result delete(Long id) { ... }
```

Spring MVC的处理流程

Spring MVC 的工作流程如下：

1. 用户发送请求，请求首先到达前端控制器 **DispatcherServlet**。
2. DispatcherServlet 接收到请求后，调用处理器映射器 **HandlerMapping**。
3. HandlerMapping 根据请求的 URL 查找对应的处理器，并生成处理器执行链 **HandlerExecutionChain**（其中包含处理器对象和相关拦截器），然后返回给 DispatcherServlet。
4. DispatcherServlet 根据找到的处理器，获取对应的处理器适配器 **HandlerAdapter**。
5. HandlerAdapter 对请求进行一系列预处理操作，例如参数封装、数据格式转换、数据校验等。
6. HandlerAdapter 调用具体的处理器 **Handler（Controller，即控制器）** 执行业务逻辑。
7. 处理器执行完成后，返回 **ModelAndView**。
8. HandlerAdapter 将 ModelAndView 返回给 DispatcherServlet。
9. DispatcherServlet 将 ModelAndView 交给视图解析器 **ViewResolver** 进行解析。
10. ViewResolver 解析后返回具体的视图 **View**。
11. DispatcherServlet 对视图进行渲染（将模型数据 Model 填充到视图中）。
12. DispatcherServlet 将最终结果响应给用户。



HandlerMapping和HandlerAdapter有了解吗?

HandlerMapping：
根据请求的 URL、参数等信息，找到并返回对应的处理器（Controller）。

HandlerAdapter：
根据处理器的类型，选择合适的方式调用 Controller，并完成参数封装、数据转换等操作。

为什么使用Spring Boot?

简化开发：
Spring Boot 提供了一系列开箱即用的组件和自动配置功能，大大简化了项目的配置过程，使开发者能够更加专注于业务逻辑的实现，而不必花费过多时间处理繁琐的框架配置。

快速启动：
Spring Boot 内置了 Tomcat、Jetty、Undertow 等 Web 服务器，应用程序可以直接运行，无需额外部署到外部服务器，从而提高了开发和测试的效率。

自动配置：
Spring Boot 基于项目的依赖和约定的规则自动完成相关配置，降低了配置的复杂度，使开发更加高效，也更容易实现最佳实践。

Spring Boot相比Spring的优势

Spring Boot 相比 Spring 的优势：

- **自动配置，简化开发：**
  Spring Boot 提供了自动配置机制，遵循“约定优于配置”的原则，大幅减少了繁琐的 XML 或 Java 配置，使开发者能够更加专注于业务逻辑的实现。
- **Starter 依赖，快速集成：**
  Spring Boot 通过提供多种 Starter 启动器，只需引入相应依赖即可快速集成常用技术（如数据库、消息队列、Web 开发等），显著提升了开发效率。
- **内嵌服务器，便于部署：**
  Spring Boot 默认集成了 Tomcat、Jetty、Undertow 等内嵌 Web 服务器，应用可以直接打包为可执行的 JAR 文件运行，无需额外部署到外部服务器，部署更加简单方便。

SpringBoot 用到哪些设计模式?

- **代理模式**：Spring 的 AOP 通过动态代理实现方法级别的切面增强，有静态和动态两种代理方式，采用动态代理方式。
- **策略模式**：Spring AOP 支持 JDK 和 Cglib 两种动态代理实现方式，通过策略接口和不同策略类，运行时动态选择，其创建一般通过工厂方法实现。
- **单例模式**：Spring Bean 默认是单例模式，通过单例注册表（如 HashMap）实现。

- **简单工厂模式**：Spring 中的 BeanFactory 是简单工厂模式的体现，通过工厂类方法获取 Bean 实例。
- **工厂方法模式**：Spring 中的 FactoryBean 体现工厂方法模式，为不同产品提供不同工厂。
- **观察者模式**：Spring 观察者模式包含 Event 事件、Listener 监听者、Publisher 发送者，通过定义事件、监听者和发送者实现，观察者注册在 ApplicationContext 中，消息发送由 ApplicationEventMulticaster 完成。
- **模板模式**：Spring Bean 的创建过程涉及模板模式，体现扩展性，类似 Callback 回调实现方式。
- **适配器模式**：Spring MVC 中针对不同方式定义的 Controller，利用适配器模式统一函数定义，定义了统一接口 HandlerAdapter 及对应适配器类。

如何理解 Spring Boot 中的“约定大于配置”?

约定大于配置是 Spring Boot 的核心理念，即通过提供合理的默认配置和统一的开发规范，减少开发者手动配置的工作量，从而提高开发效率和项目的规范性。

主要体现在以下几个方面：

- **自动配置：**
  Spring Boot 会根据项目的依赖和运行环境自动完成大部分配置，例如引入 `spring-boot-starter-web` 后，会自动配置内嵌 Tomcat 和 Spring MVC，无需手动编写大量配置。
- **默认配置：**
  框架提供了丰富的默认设置，如日志、Web 服务器、数据源等，开发者只需在需要时进行个性化修改。
- **统一规范：**
  Spring Boot 约定了项目结构和开发方式，使项目更规范，方便团队协作和维护。

Spring Boot的项目结构

- 终端显示层（前后端分离的话没有这个）
- 开放接口层：可以直接封装Service层接口暴露成rpc接口
- 请求处理层（Web层）：主要对访问控制进行转发、各类基本参数校验、不复用的简单业务处理
- 业务逻辑层（Service层）：处理具体的业务逻辑
- 通用逻辑层（Manager层）：
  - 对第三方平台封装的层，预处理返回结果及转换异常信息，适配上层接口
  - 对Service层通用能力的下沉，如缓存方案、中间件通用处理
  - 与DAO层交互，对多个DAO的组合复用
- 数据持久层（DAO层）：负责数据库操作
- 第三方服务：包括其他部门RPC服务接口，其他公司的HTTP接口等

什么是自动装配?

AutoConfigurationImportSelector的主要工作?

在应用程序启动时，`AutoConfigurationImportSelector`会扫描类路径下的META-INFO/spring.factories文件，这个文件中包含了各种Spring配置，他会查找所有实现了`AutoConfiguration`接口的类，对于每一个自动配置类，`AutoConfigurationImportSelector`会使用条件判断机制确认这个类是否满足导入条件，满足条件的自动配置类会被导入到应用程序的上下文中，这意味着它们将会被实例化并应用于应用程序的配置

事务的传播方式

REQUIRED：如果当前没有事务，则新建事务，否则加入已有事务

REQUIRED_NEW：新建事务，如果当前存在事务就把当前事务挂起

NESTED：如果当前没有事务，则新建事务，否则创建嵌套子事务

REQUIRED和NESTED的区别是：如果内部方法异常，REQUIRED会使得整个事务一起回滚，而NESTED只会回滚到savepoint

SUPPORTS：支持当前事务，如果当前没有事务，则以非事务方式运行

NOT_SUPPORTS：以非事务方式运行，如果当前存在事务，就把当前事务挂起

MANDATORY：使用当前事务，如果当前不存在事务则抛出异常

NEVER：以非事务方式运行，如果存在事务则抛出异常

多态解决了什么问题?

多态提高了代码的扩展性和复用性，它是许多设计模式、设计原则、编程技巧的代码实现基础，比如策略模式，里氏替换原则，依赖倒置原则，基于接口而非实现编程，利用多态去掉冗长的if-else语句等等

这四个概念是面向对象设计（OOD）的灵魂，它们彼此交织，共同指向一个核心目标：**解耦。**

正如你敏锐察觉到的，**多态（Polymorphism）**是支撑这些高层建筑的地基。如果没有多态，这些原则和模式都只是空中楼阁。

---

1. 核心概念解析

策略模式 (Strategy Pattern)

策略模式定义了一系列算法，并将每个算法封装起来，使它们可以相互替换。它让算法的变化独立于使用算法的客户。

- **形象理解：** 你的导航 App 提供了“驾车”、“公交”、“步行”三种方案。App 并不关心具体路是怎么算的，它只负责调用 `calculateRoute()`。你切换出行方式，就是在切换“策略”。

基于接口而非实现编程 (Program to an Interface, not an Implementation)

这是设计模式的基石。它要求我们在声明变量、参数或返回类型时，尽量使用**抽象类或接口**，而不是具体的实现类。

- **形象理解：** 你买插头时，只要它是“国标三孔”，你就能用。你不需要知道这个插头是哪家工厂生产的，你只对接“接口标准”。

依赖倒置原则 (Dependency Inversion Principle, DIP)

高层模块不应依赖低层模块，二者都应依赖抽象；抽象不应依赖细节，细节应依赖抽象。

- **核心内涵：** 以前是“高层 -> 低层”，现在是“高层 -> 抽象 <- 低层”。通过引入抽象层，反转了依赖关系，从而实现了组件的可插拔。

里式替换原则 (Liskov Substitution Principle, LSP)

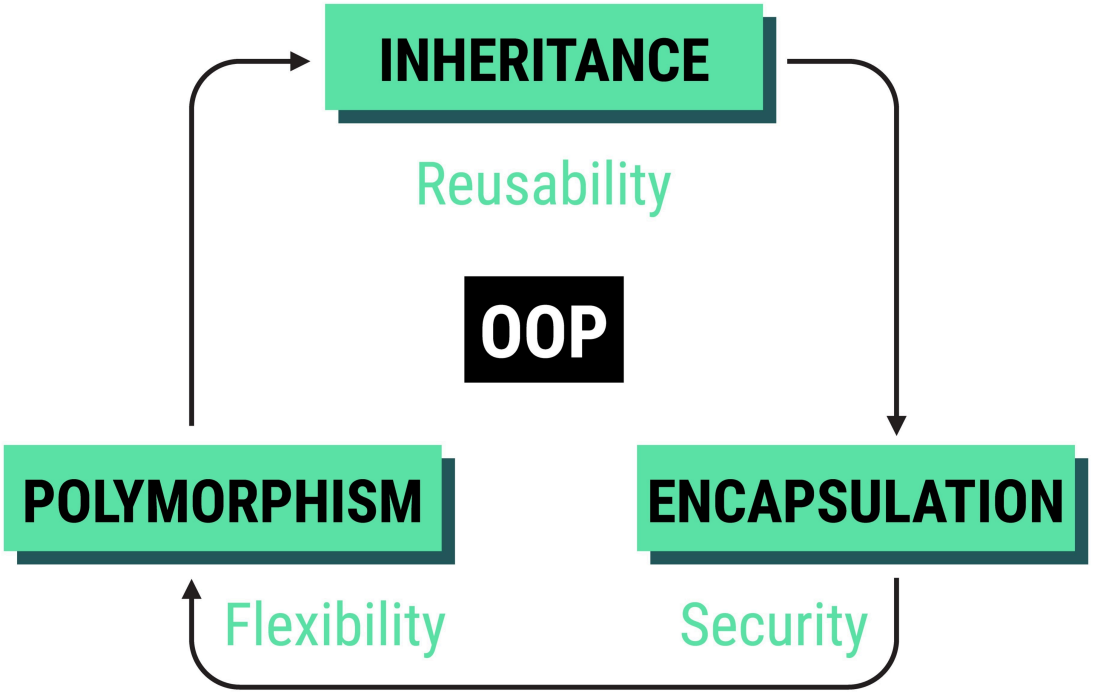
子类对象必须能够替换掉程序中任意出现的父类对象，并且保证原有程序的逻辑行为不变。

- **形象理解：** 如果你发现一个“正方形”继承了“长方形”，但在设置宽高时导致程序逻辑崩溃（比如面积计算不对），那么这个继承就违反了 LSP。子类不仅要承接父类的“形”，还要承接父类的“意”。

---

2. 为什么它们的基础全是“多态”？

多态是指**“同一个行为，在不同对象上有不同的表现形式”**。它是实现“运行时替换”的技术手段。



Shutterstock

我们来看看多态是如何支撑这些原则的：

1. 多态提供了“替换的能力”

- **对应原则：里式替换原则 (LSP)**
- **逻辑：** 多态允许我们将子类引用赋值给父类变量。LSP 在语义层面规定了这种替换必须是“安全”的。如果没有多态的语法支持，你根本无法在代码中进行替换；如果没有 LSP 的逻辑约束，你的多态替换就会导致 Bug。

2. 多态实现了“抽象的具象化”

- **对应原则：基于接口而非实现、依赖倒置 (DIP)**
- **逻辑：** 当我们说 `Shape s = new Circle();` 时，`Shape` 是接口，`Circle` 是实现。
  - **接口编程**让你敢于写下 `s.draw()`，因为它相信多态会在运行时找到正确的实现。
  - **依赖倒置**让你敢于让高层模块只拿住一个 `Shape` 引用，而不去管具体的 `Circle` 或 `Square`，因为多态机制会自动完成“细节对抽象的填充”。

3. 多态是“动态切换”的引擎

- **对应模式：策略模式**
- **逻辑：** 策略模式的核心代码通常长这样：

  Java

  ```
  public void executeStrategy(Strategy s) {
      s.run(); // 这里就是多态！运行时传入哪个子类，就跑哪个算法。
  }
  ```

  如果没有多态，你必须用大量的 `if-else` 或 `switch` 来判断当前是什么策略。多态消除了这些硬编码，让程序在运行时通过**动态绑定**来决定行为。

总结

概念	侧重点	与多态的关系
基于接口编程	**编码习惯**	多态是其存在的**前提**
里式替换原则	**继承规范**	为多态的安全性**定规矩**
依赖倒置原则	**架构方向**	利用多态实现**解耦**
策略模式	**具体方案**	是多态在算法切换上的**典型应用**

一句话总结： 多态是 OOD 的**物理基础**（工具），而这三原则一模式是利用多态进行设计的**进阶指南**（方法论）。

既然聊到了多态，你是更关心它在 Java/C++ 这类静态语言中的实现原理（如虚函数表），还是更关心如何在实际项目中重构掉繁琐的 `if-else`？